

Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 22: Classification Assessment

Classification Assessment

A classifier is a model or function M that predicts the class label $\hat{y}$ for a given input example $\mathbf{x}$:

$$\hat{y} = M(\mathbf{x})$$

where $\mathbf{x} = (x_1, x_2, \dots, x_d)^T \in \mathbb{R}^d$ is a point in d -dimensional space and $\hat{y} \in \{c_1, c_2, \dots, c_k\}$ is its predicted class.

To build the classification model M we need a *training set* of points along with their known classes.

Once the model M has been trained, we assess its performance over a separate *testing set* of points for which we know the true classes.

Finally, the model can be deployed to predict the class for future points whose class we typically do not know.

Classification Performance Measures

Let $\mathbf{D}$ be the testing set comprising n points in a d -dimensional space, let $\{c_1, c_2, \dots, c_k\}$ denote the set of k class labels, and let M be a classifier. For $\mathbf{x}_i \in \mathbf{D}$, let y_i denote its true class, and let $\hat{y}_i = M(\mathbf{x}_i)$ denote its predicted class.

Error Rate: The error rate is the fraction of incorrect predictions for the classifier over the testing set, defined as

$$\text{Error Rate} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i)$$

where I is an indicator function. Error rate is an estimate of the probability of misclassification. The lower the error rate the better the classifier.

Accuracy: The accuracy of a classifier is the fraction of correct predictions:

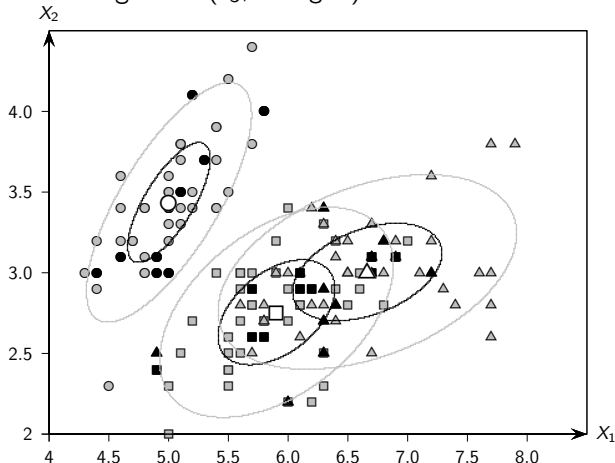
$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n I(y_i = \hat{y}_i) = 1 - \text{Error Rate}$$

Accuracy gives an estimate of the probability of a correct prediction; thus, the higher the accuracy, the better the classifier.

Iris Data: Full Bayes Classifier

Training data in grey. Testing data in black.

Three Classes: Iris-setosa (c_1 ; circles), Iris-versicolor (c_2 ; squares) and Iris-virginica (c_3 ; triangles)



Mean (in white) and density contours (1 and 2 standard deviations) shown for each class.

The classifier misclassifies 8 out of the 30 test cases. Thus, we have

$$\text{Error Rate} = \frac{08}{30} = 0.27$$

$$\text{Accuracy} = \frac{22}{30} = 0.73$$

Contingency Table-based Measures

Let $\mathcal{D} = \{\mathbf{D}_1, \mathbf{D}_2, \dots, \mathbf{D}_k\}$ denote a partitioning of the testing points based on their true class labels, where $\mathbf{D}_j = \{\mathbf{x}_i \in \mathbf{D} \mid y_i = c_j\}$. Let $n_i = |\mathbf{D}_i|$ denote the size of true class c_i .

Let $\mathcal{R} = \{\mathbf{R}_1, \mathbf{R}_2, \dots, \mathbf{R}_k\}$ denote a partitioning of the testing points based on the predicted labels, that is, $\mathbf{R}_j = \{\mathbf{x}_i \in \mathbf{D} \mid \hat{y}_i = c_j\}$. Let $m_j = |\mathbf{R}_j|$ denote the size of the predicted class c_j .

$\mathcal{R}$ and $\mathcal{D}$ induce a $k \times k$ contingency table $\mathbf{N}$, also called a *confusion matrix*, defined as follows:

$$\mathbf{N}(i, j) = n_{ij} = |\mathbf{R}_i \cap \mathbf{D}_j| = \left| \{ \mathbf{x}_a \in \mathbf{D} \mid \hat{y}_a = c_i \text{ and } y_a = c_j \} \right|$$

where $1 \leq i, j \leq k$. The count n_{ij} denotes the number of points with predicted class c_i whose true label is c_j . Thus, n_{ii} (for $1 \leq i \leq k$) denotes the number of cases where the classifier agrees on the true label c_i . The remaining counts n_{ij} , with $i \neq j$, are cases where the classifier and true labels disagree.

Accuracy/Precision and Coverage/Recall

The class-specific *accuracy* or *precision* of the classifier M for class c_i is given as the fraction of correct predictions over all points predicted to be in class c_i

$$acc_i = prec_i = \frac{n_{ii}}{m_i}$$

where m_i is the number of examples predicted as c_i by classifier M . The higher the accuracy on class c_i the better the classifier. The overall precision or accuracy of the classifier is the weighted average of class-specific accuracies:

$$Accuracy = Precision = \sum_{i=1}^k \left(\frac{m_i}{n} \right) acc_i = \frac{1}{n} \sum_{i=1}^k n_{ii}$$

The class-specific *coverage* or *recall* of M for class c_i is the fraction of correct predictions over all points in class c_i :

$$coverage_i = recall_i = \frac{n_{ii}}{n_i}$$

The higher the coverage the better the classifier.

F-measure

The *class-specific F-measure* tries to balance the precision and recall values, by computing their harmonic mean for class c_i :

$$F_i = \frac{2}{\frac{1}{\text{prec}_i} + \frac{1}{\text{recall}_i}} = \frac{2 \cdot \text{prec}_i \cdot \text{recall}_i}{\text{prec}_i + \text{recall}_i} = \frac{2 n_{ii}}{n_i + m_i}$$

The higher the F_i value the better the classifier.

The overall *F-measure* for the classifier M is the mean of the class-specific values:

$$F = \frac{1}{k} \sum_{i=1}^r F_i$$

For a perfect classifier, the maximum value of the F-measure is 1.

Contingency Table for Iris: Full Bayes Classifier

Predicted	True			
	Iris-setosa (c_1)	Iris-versicolor (c_2)	Iris-virginica (c_3)	
Iris-setosa (c_1)	10	0	0	$m_1 = 10$
Iris-versicolor (c_2)	0	7	5	$m_2 = 12$
Iris-virginica (c_3)	0	3	5	$m_3 = 8$
	$n_1 = 10$	$n_2 = 10$	$n_3 = 10$	$n = 30$

The class-specific precision, recall and F-measure values are:

$$prec_1 = \frac{n_{11}}{m_1} = \frac{10}{10} = 1.0 \quad recall_1 = \frac{n_{11}}{n_1} = \frac{10}{10} = 1.0 \quad F_1 = \frac{2 \cdot n_{11}}{(n_1 + m_1)} = \frac{20}{20} = 1.0$$

$$prec_2 = \frac{n_{22}}{m_2} = \frac{7}{12} = 0.583 \quad recall_2 = \frac{n_{22}}{n_2} = \frac{7}{10} = 0.7 \quad F_2 = \frac{2 \cdot n_{22}}{(n_2 + m_2)} = \frac{14}{22} = 0.636$$

$$prec_3 = \frac{n_{33}}{m_3} = \frac{5}{8} = 0.625 \quad recall_3 = \frac{n_{33}}{n_3} = \frac{5}{10} = 0.5 \quad F_3 = \frac{2 \cdot n_{33}}{(n_3 + m_3)} = \frac{10}{18} = 0.556$$

The overall accuracy and F-measure is

$$Accuracy = \frac{(n_{11} + n_{22} + n_{33})}{n} = \frac{(10 + 7 + 5)}{30} = 22/30 = 0.733$$

$$F = \frac{1}{3}(1.0 + 0.636 + 0.556) = \frac{2.192}{3} = 0.731$$

Binary Classification: Positive and Negative Class

When there are only $k = 2$ classes, we call class c_1 the positive class and c_2 the negative class. The entries of the resulting 2×2 confusion matrix are

Predicted Class	True Class	
	Positive (c_1)	Negative (c_2)
Positive (c_1)	True Positive (TP)	False Positive (FP)
Negative (c_2)	False Negative (FN)	True Negative (TN)

Binary Classification: Positive and Negative Class

- *True Positives (TP)*: The number of points that the classifier correctly predicts as positive:

$$TP = n_{11} = |\{\mathbf{x}_i \mid \hat{y}_i = y_i = c_1\}|$$

- *False Positives (FP)*: The number of points the classifier predicts to be positive, which in fact belong to the negative class:

$$FP = n_{12} = |\{\mathbf{x}_i \mid \hat{y}_i = c_1 \text{ and } y_i = c_2\}|$$

- *False Negatives (FN)*: The number of points the classifier predicts to be in the negative class, which in fact belong to the positive class:

$$FN = n_{21} = |\{\mathbf{x}_i \mid \hat{y}_i = c_2 \text{ and } y_i = c_1\}|$$

- *True Negatives (TN)*: The number of points that the classifier correctly predicts as negative:

$$TN = n_{22} = |\{\mathbf{x}_i \mid \hat{y}_i = y_i = c_2\}|$$

Binary Classification: Assessment Measures

Error Rate: The error rate for the binary classification case is given as the fraction of mistakes (or false predictions):

$$\text{Error Rate} = \frac{FP + FN}{n}$$

Accuracy: The accuracy is the fraction of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{n}$$

The precision for the positive and negative class is given as

$$\begin{aligned} \text{prec}_P &= \frac{TP}{TP + FP} = \frac{TP}{m_1} \\ \text{prec}_N &= \frac{TN}{TN + FN} = \frac{TN}{m_2} \end{aligned}$$

where $m_i = |\mathbf{R}_i|$ is the number of points predicted by M as having class c_i .

Sensitivity or True Positive Rate: The fraction of correct predictions with respect to all points in the positive class, i.e., the recall for the positive class

$$TPR = sensitivity = recall_P = \frac{TP}{TP + FN} = \frac{TP}{n_1}$$

where n_1 is the size of the positive class.

Specificity or True Negative Rate: The recall for the negative class:

$$TNR = specificity = recall_N = \frac{TN}{FP + TN} = \frac{TN}{n_2}$$

where n_2 is the size of the negative class.

False Negative Rate: Defined as

$$FNR = \frac{FN}{TP + FN} = \frac{FN}{n_1} = 1 - \textit{sensitivity}$$

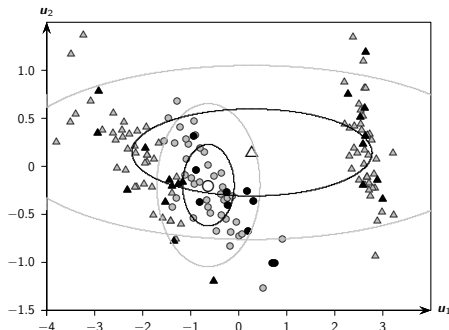
False Positive Rate: Defined as

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{n_2} = 1 - \textit{specificity}$$

Iris Principal Components Data: Naive Bayes Classifier

Iris-versicolor (c_1 - circles) and other two Irises (c_2 - triangles).

Training data (80%) in grey and testing data (20%) in black.



$$\hat{P}(c_1) = 40/120 = 0.33$$

$$\hat{\mu}_1 = (-0.641 \quad -0.204)^T$$

$$\hat{\Sigma}_1 = \begin{pmatrix} 0.29 & 0 \\ 0 & 0.18 \end{pmatrix}$$

$$\hat{P}(c_2) = 80/120 = 0.67$$

$$\hat{\mu}_2 = (0.27 \quad 0.14)^T$$

$$\hat{\Sigma}_2 = \begin{pmatrix} 6.14 & 0 \\ 0 & 0.206 \end{pmatrix}$$

The mean (in white) and the contour plot of the normal distribution for each class are shown; the contours are shown for one and two standard deviations along each axis.

Iris PC Data: Assessment Measures

	True		
Predicted	Positive (c_1)	Negative (c_2)	
Positive (c_1)	$TP = 7$	$FP = 7$	$m_1 = 14$
Negative (c_2)	$FN = 3$	$TN = 13$	$m_2 = 16$
	$n_1 = 10$	$n_2 = 20$	$n = 30$

The naive Bayes classifier misclassified 10 out of the 30 test instances, resulting in an error rate and accuracy of

$$\text{Error Rate} = 10/30 = 0.33$$

$$\text{Accuracy} = 20/30 = 0.67$$

Other performance measures:

$$\text{prec}_P = \frac{TP}{TP + FP} = \frac{7}{14} = 0.5$$

$$\text{prec}_N = \frac{TN}{TN + FN} = \frac{13}{16} = 0.8125$$

$$\text{recall}_P(\text{sensitivity}) = \frac{TP}{TP + FN} = \frac{7}{10} = 0.7$$

$$\text{recall}_N = \text{specificity} = \frac{TN}{TN + FP} = \frac{13}{20} = 0.65$$

$$\text{FNR} = 1 - \text{sensitivity} = 0.3$$

$$\text{FPR} = 1 - \text{specificity} = 0.35$$

ROC Analysis

Receiver Operating Characteristic (ROC) analysis is a popular strategy for assessing the performance of classifiers when there are two classes.

ROC analysis requires that a classifier output a score value for the positive class for each point in the testing set. These scores can then be used to order points in decreasing order.

Typically, a binary classifier chooses some positive score threshold ρ , and classifies all points with score above ρ as positive, with the remaining points classified as negative.

ROC analysis plots the performance of the classifier over all possible values of the threshold parameter ρ .

In particular, for each value of ρ , it plots the false positive rate (1-specificity) on the x-axis versus the true positive rate (sensitivity) on the y-axis. The resulting plot is called the *ROC curve* or *ROC plot* for the classifier.

ROC Analysis

Let $S(\mathbf{x}_i)$ denote the real-valued score for the positive class output by a classifier M for the point $\mathbf{x}_i$. Let the maximum and minimum score thresholds observed on testing dataset $\mathbf{D}$ be as follows:

$$\rho^{\min} = \min_i \{S(\mathbf{x}_i)\}$$

$$\rho^{\max} = \max_i \{S(\mathbf{x}_i)\}$$

Initially, we classify all points as negative. Both TP and FP are thus initially zero, as given in the confusion matrix:

Predicted	True	
	Pos	Neg
Pos	0	0
Neg	FN	TN

This results in TPR and FPR rates of zero, which correspond to the point $(0,0)$ at the lower left corner in the ROC plot.

Next, for each distinct value of ρ in the range $[\rho^{\min}, \rho^{\max}]$, we tabulate the set of positive points:

$$R_1(\rho) = \{\mathbf{x}_i \in \mathbf{D} : S(\mathbf{x}_i) > \rho\}$$

and we compute the corresponding true and false positive rates, to obtain a new point in the ROC plot.

Finally, in the last step, we classify all points as positive. Both FN and TN are thus zero, as per the confusion matrix

	True	
Predicted	Pos	Neg
Pos	TP	FP
Neg	0	0

resulting in TPR and FPR values of 1. This results in the point $(1,1)$ at the top right-hand corner in the ROC plot.

ROC Analysis

An ideal classifier corresponds to the top left point $(0,1)$, which corresponds to the case $FPR = 0$ and $TPR = 1$, that is, the classifier has no false positives, and identifies all true positives (as a consequence, it also correctly predicts all the points in the negative class). This case is shown in the confusion matrix:

Predicted	True	
	Pos	Neg
Pos	TP	0
Neg	0	TN

A classifier with a curve closer to the ideal case, that is, closer to the upper left corner, is a better classifier.

Area Under ROC Curve: The area under the ROC curve, abbreviated AUC, can be used as a measure of classifier performance. The AUC value is essentially the probability that the classifier will rank a random positive test case higher than a random negative test instance.

ROC: Different Cases for 2×2 Confusion Matrix

Predicted	True	
	Pos	Neg
Pos	0	0
Neg	<i>FN</i>	<i>TN</i>

(a) Initial: all negative

Predicted	True	
	Pos	Neg
Pos	<i>TP</i>	<i>FP</i>
Neg	0	0

(b) Final: all positive

Predicted	True	
	Pos	Neg
Pos	<i>TP</i>	0
Neg	0	<i>TN</i>

(c) Ideal classifier

Random Classifier

A random classifier corresponds to a diagonal line in the ROC plot.

Consider a classifier that randomly guesses the class of a point as positive half the time, and negative the other half. We then expect that half of the true positives and true negatives will be identified correctly, resulting in the point $(TPR, FPR) = (0.5, 0.5)$ for the ROC plot.

In general, any fixed probability of prediction, say r , for the positive class, yields the point (r, r) in ROC space.

The diagonal line thus represents the performance of a random classifier, over all possible positive class prediction thresholds r .

ROC/AUC Algorithm

The ROC/AUC takes as input the testing set $\mathbf{D}$, and the classifier M .

The first step is to predict the score $S(\mathbf{x}_i)$ for the positive class (c_1) for each test point $\mathbf{x}_i \in \mathbf{D}$. Next, we sort the $(S(\mathbf{x}_i), y_i)$ pairs, that is, the score and the true class pairs, in decreasing order of the scores

Initially, we set the positive score threshold $\rho = \infty$. We then examine each pair $(S(\mathbf{x}_i), y_i)$ in sorted order, and for each distinct value of the score, we set $\rho = S(\mathbf{x}_i)$ and plot the point

$$(FPR, TPR) = \left(\frac{FP}{n_2}, \frac{TP}{n_1} \right)$$

As each test point is examined, the true and false positive values are adjusted based on the true class y_i for the test point $\mathbf{x}_i$. If $y_1 = c_1$, we increment the true positives, otherwise, we increment the false positives

ROC/AUC Algorithm

The AUC value is computed as each new point is added to the ROC plot. The algorithm maintains the previous values of the false and true positives, FP_{prev} and TP_{prev} , for the previous score threshold ρ .

Given the current FP and TP values, we compute the area under the curve defined by the four points

$$\begin{aligned}(x_1, y_1) &= \left(\frac{FP_{prev}}{n_2}, \frac{TP_{prev}}{n_1} \right) & (x_2, y_2) &= \left(\frac{FP}{n_2}, \frac{TP}{n_1} \right) \\ (x_1, 0) &= \left(\frac{FP_{prev}}{n_2}, 0 \right) & (x_2, 0) &= \left(\frac{FP}{n_2}, 0 \right)\end{aligned}$$

These four points define a trapezoid, whenever $x_2 > x_1$ and $y_2 > y_1$, otherwise, they define a rectangle (which may be degenerate, with zero area).

The area under the trapezoid is given as $b \cdot h$, where $b = |x_2 - x_1|$ is the length of the base of the trapezoid and $h = \frac{1}{2}(y_2 + y_1)$ is the average height of the trapezoid.

Algorithm ROC-Curve

ROC-Curve(D, M):

```
1  $n_1 \leftarrow |\{x_i \in D | y_i = c_1\}|$  // size of positive class
2  $n_2 \leftarrow |\{x_i \in D | y_i = c_2\}|$  // size of negative class
   // classify, score, and sort all test points
3  $L \leftarrow$  sort the set  $\{(S(x_i), y_i) : x_i \in D\}$  by decreasing scores
4  $FP \leftarrow TP \leftarrow 0$ 
5  $FP_{prev} \leftarrow TP_{prev} \leftarrow 0$ 
6  $AUC \leftarrow 0$ 
7  $\rho \leftarrow \infty$ 
8 foreach  $(S(x_i), y_i) \in L$  do
9   if  $\rho > S(x_i)$  then
10     plot point  $\left(\frac{FP}{n_2}, \frac{TP}{n_1}\right)$ 
11      $AUC \leftarrow AUC + \text{Trapezoid-Area}\left(\left(\frac{FP_{prev}}{n_2}, \frac{TP_{prev}}{n_1}\right), \left(\frac{FP}{n_2}, \frac{TP}{n_1}\right)\right)$ 
12      $\rho \leftarrow S(x_i)$ 
13      $FP_{prev} \leftarrow FP$ 
14      $TP_{prev} \leftarrow TP$ 
15   if  $y_i = c_1$  then  $TP \leftarrow TP + 1$ 
16   else  $FP \leftarrow FP + 1$ 
17 plot point  $\left(\frac{FP}{n_2}, \frac{TP}{n_1}\right)$ 
18  $AUC \leftarrow AUC + \text{Trapezoid-Area}\left(\left(\frac{FP_{prev}}{n_2}, \frac{TP_{prev}}{n_1}\right), \left(\frac{FP}{n_2}, \frac{TP}{n_1}\right)\right)$ 
```

Algorithm Trapezoid-Area

Trapezoid-Area $((x_1, y_1), (x_2, y_2))$:

- 1 $b \leftarrow |x_2 - x_1|$ // base of trapezoid
- 2 $h \leftarrow \frac{1}{2}(y_2 + y_1)$ // average height of trapezoid
- 3 **return** $(b \cdot h)$

Iris PC Data: ROC Analysis

We use the naive Bayes classifier to compute the probability that each test point belongs to the positive class (c_1 ; iris-versicolor).

The score of the classifier for test point $\mathbf{x}_i$ is therefore $S(\mathbf{x}_i) = P(c_1|\mathbf{x}_i)$. The sorted scores (in decreasing order) along with the true class labels are as follows:

$S(\mathbf{x}_i)$	0.93	0.82	0.80	0.77	0.74	0.71	0.69	0.67	0.66	0.61
y_i	c_2	c_1	c_2	c_1	c_1	c_1	c_2	c_1	c_2	c_2

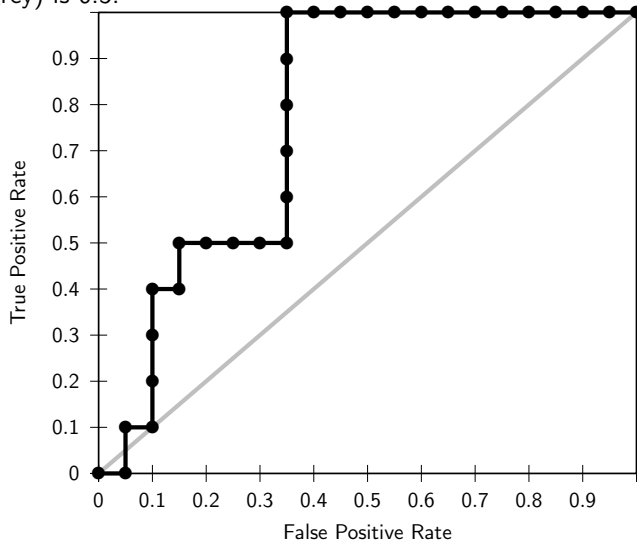
$S(\mathbf{x}_i)$	0.59	0.55	0.55	0.53	0.47	0.30	0.26	0.11	0.04	2.97e-03
y_i	c_2	c_2	c_1	c_1	c_1	c_1	c_1	c_2	c_2	c_2

$S(\mathbf{x}_i)$	1.28e-03	2.55e-07	6.99e-08	3.11e-08	3.109e-08
y_i	c_2	c_2	c_2	c_2	c_2

$S(\mathbf{x}_i)$	1.53e-08	9.76e-09	2.08e-09	1.95e-09	7.83e-10
y_i	c_2	c_2	c_2	c_2	c_2

ROC Plot for Iris PC Data

AUC for naive Bayes is 0.775, whereas the AUC for the random classifier (ROC plot in grey) is 0.5.



ROC Plot and AUC: Trapezoid Region

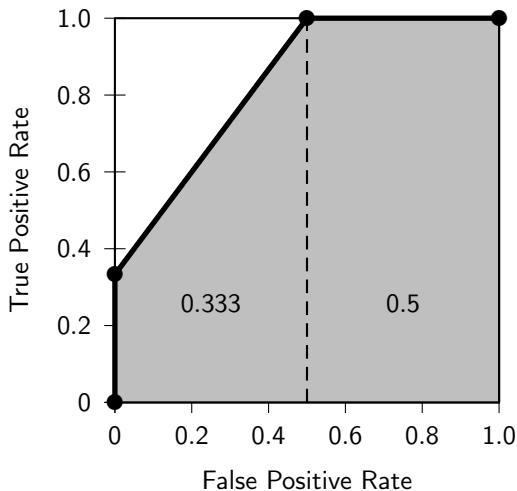
Consider the following sorted scores, along with the true class, for some testing dataset with $n = 5$, $n_1 = 3$ and $n_2 = 2$.

$$(0.9, c_1), (0.8, c_2), (0.8, c_1), (0.8, c_1), (0.1, c_2)$$

Algorithm yields the following points that are added to the ROC plot, along with the running AUC:

ρ	FP	TP	(FPR, TPR)	AUC
∞	0	0	(0,0)	0
0.9	0	1	(0,0.333)	0
0.8	1	3	(0.5,1)	0.333
0.1	2	3	(1,1)	0.833

ROC Plot and AUC: Trapezoid Region



Classifier Evaluation

Consider a classifier M , and some performance measure θ . Typically, the input dataset $\mathbf{D}$ is randomly split into a disjoint training set and testing set. The training set is used to learn the model M , and the testing set is used to evaluate the measure θ .

How confident can we be about the classification performance? The results may be due to an artifact of the random split.

Also $\mathbf{D}$ is itself a d -dimensional multivariate random sample drawn from the true (unknown) joint probability density function $f(\mathbf{x})$ that represents the population of interest. Ideally, we would like to know the expected value $E[\theta]$ of the performance measure over all possible testing sets drawn from f . However, because f is unknown, we have to estimate $E[\theta]$ from $\mathbf{D}$.

Cross-validation and resampling are two common approaches to compute the expected value and variance of a given performance measure.

K-fold Cross-Validation

Cross-validation divides the dataset $\mathbf{D}$ into K equal-sized parts, called *folds*, namely $\mathbf{D}_1, \mathbf{D}_2, \dots, \mathbf{D}_K$.

Each fold $\mathbf{D}_i$ is, in turn, treated as the testing set, with the remaining folds comprising the training set $\mathbf{D} \setminus \mathbf{D}_i = \bigcup_{j \neq i} \mathbf{D}_j$.

After training the model M_i on $\mathbf{D} \setminus \mathbf{D}_i$, we assess its performance on the testing set $\mathbf{D}_i$ to obtain the i -th estimate θ_i .

The expected value of the performance measure can then be estimated as

$$\hat{\mu}_{\theta} = E[\theta] = \frac{1}{K} \sum_{i=1}^K \theta_i$$

and its variance as

$$\hat{\sigma}_{\theta}^2 = \frac{1}{K} \sum_{i=1}^K (\theta_i - \hat{\mu}_{\theta})^2$$

Usually K is chosen to be 5 or 10. The special case, when $K = n$, is called *leave-one-out* cross-validation.

K-fold Cross-Validation Algorithm

Cross-Validation(K, D):

- 1 $D \leftarrow$ randomly shuffle D
- 2 $\{D_1, D_2, \dots, D_K\} \leftarrow$ partition D in K equal parts
- 3 **foreach** $i \in [1, K]$ **do**
- 4 $M_i \leftarrow$ train classifier on $D \setminus D_i$
- 5 $\theta_i \leftarrow$ assess M_i on D_i
- 6 $\hat{\mu}_\theta = \frac{1}{K} \sum_{i=1}^K \theta_i$
- 7 $\hat{\sigma}_\theta^2 = \frac{1}{K} \sum_{i=1}^K (\theta_i - \hat{\mu}_\theta)^2$
- 8 **return** $\hat{\mu}_\theta, \hat{\sigma}_\theta^2$

2D Iris Dataset: K -fold Cross-Validation

Consider the 2D Iris dataset with $k = 3$ classes. We assess the error rate of the full Bayes classifier via 5-fold cross-validation, obtaining the following error rates when testing on each fold:

$$\theta_1 = 0.267 \quad \theta_2 = 0.133 \quad \theta_3 = 0.233 \quad \theta_4 = 0.367 \quad \theta_5 = 0.167$$

The mean and variance for the error rate are as follows:

$$\hat{\mu}_\theta = \frac{1.167}{5} = 0.233 \qquad \hat{\sigma}_\theta^2 = 0.00833$$

Performing ten 5-fold cross-validation runs for the Iris dataset results in the mean of the expected error rate as 0.232, and the mean of the variance as 0.00521, with the variance in both these estimates being less than 10^{-3} .

Bootstrap Resampling

The bootstrap method draws K random samples of size n *with replacement* from $\mathbf{D}$. Each sample $\mathbf{D}_i$ is thus the same size as $\mathbf{D}$, and has several repeated points.

The probability that a particular point $\mathbf{x}_j$ is not selected even after n tries is given as

$$P(\mathbf{x}_j \notin \mathbf{D}_i) = q^n = \left(1 - \frac{1}{n}\right)^n \simeq e^{-1} = 0.368$$

which implies that each bootstrap sample contains approximately 63.2% of the points from $\mathbf{D}$.

The bootstrap samples can be used to evaluate the classifier by training it on each of samples $\mathbf{D}_i$ and then using the full input dataset $\mathbf{D}$ as the testing set.

However, the estimated mean and variance of θ will be somewhat optimistic owing to the fairly large overlap between the training and testing datasets (63.2%).

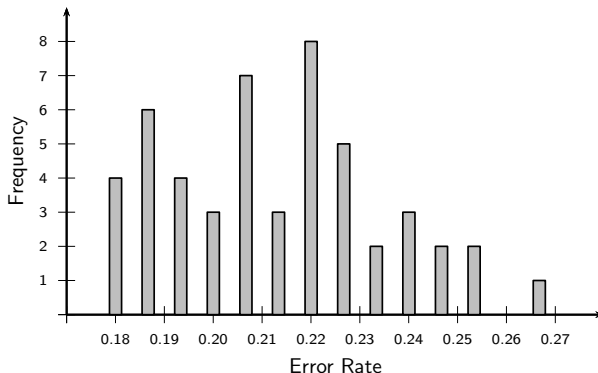
Bootstrap Resampling Algorithm

Bootstrap-Resampling(K, D):

- 1 **for** $i \in [1, K]$ **do**
- 2 $D_i \leftarrow$ sample of size n with replacement from D
- 3 $M_i \leftarrow$ train classifier on D_i
- 4 $\theta_i \leftarrow$ assess M_i on D
- 5 $\hat{\mu}_\theta = \frac{1}{K} \sum_{i=1}^K \theta_i$
- 6 $\hat{\sigma}_\theta^2 = \frac{1}{K} \sum_{i=1}^K (\theta_i - \hat{\mu}_\theta)^2$
- 7 **return** $\hat{\mu}_\theta, \hat{\sigma}_\theta^2$

Iris 2D Data: Bootstrap Resampling using Error Rate

We apply bootstrap sampling to estimate the error rate for the full Bayes classifier, using $K = 50$ samples. The sampling distribution of error rates is:



The expected value and variance of the error rate are

$$\hat{\mu}_{\theta} = 0.213$$

$$\hat{\sigma}_{\theta}^2 = 4.815 \times 10^{-4}$$

Confidence Intervals

We would like to derive confidence bounds on how much the estimated mean and variance may deviate from the true value.

To answer this question we make use of the central limit theorem, which states that the sum of a large number of independent and identically distributed (IID) random variables has approximately a normal distribution, regardless of the distribution of the individual random variables.

Let $\theta_1, \theta_2, \dots, \theta_K$ be a sequence of IID random variables, representing, for example, the error rate or some other performance measure over the K -folds in cross-validation or K bootstrap samples.

Assume that each θ_i has a finite mean $E[\theta_i] = \mu$ and finite variance $\text{var}(\theta_i) = \sigma^2$.

Let $\hat{\mu}$ denote the sample mean:

$$\hat{\mu} = \frac{1}{K}(\theta_1 + \theta_2 + \dots + \theta_K)$$

Confidence Intervals

By linearity of expectation, we have

$$E[\hat{\mu}] = E\left[\frac{1}{K}(\theta_1 + \theta_2 + \dots + \theta_K)\right] = \frac{1}{K} \sum_{i=1}^K E[\theta_i] = \frac{1}{K} (K\mu) = \mu$$

The variance of $\hat{\mu}$ is given as

$$\text{var}(\hat{\mu}) = \text{var}\left(\frac{1}{K}(\theta_1 + \theta_2 + \dots + \theta_K)\right) = \frac{1}{K^2} \sum_{i=1}^K \text{var}(\theta_i) = \frac{1}{K^2} (K\sigma^2) = \frac{\sigma^2}{K}$$

Thus, the standard deviation of $\hat{\mu}$ is given as

$$\text{std}(\hat{\mu}) = \sqrt{\text{var}(\hat{\mu})} = \frac{\sigma}{\sqrt{K}}$$

We are interested in the distribution of the z-score of $\hat{\mu}$, which is itself a random variable

$$Z_K = \frac{\hat{\mu} - E[\hat{\mu}]}{\text{std}(\hat{\mu})} = \frac{\hat{\mu} - \mu}{\frac{\sigma}{\sqrt{K}}} = \sqrt{K} \left(\frac{\hat{\mu} - \mu}{\sigma} \right)$$

Z_K specifies the deviation of the estimated mean from the true mean in terms of its standard deviation.

Confidence Intervals

The central limit theorem states that as the sample size increases, the random variable Z_K *converges in distribution* to the standard normal distribution (which has mean 0 and variance 1). That is, as $K \rightarrow \infty$, for any $x \in \mathbb{R}$, we have

$$\lim_{K \rightarrow \infty} P(Z_K \leq x) = \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function for the standard normal density function $f(x|0,1)$.

Let $z_{\alpha/2}$ denote the z-score value that encompasses $\alpha/2$ of the probability mass for a standard normal distribution, that is,

$$P(0 \leq Z_K \leq z_{\alpha/2}) = \Phi(z_{\alpha/2}) - \Phi(0) = \alpha/2$$

then, because the normal distribution is symmetric about the mean, we have

$$\lim_{K \rightarrow \infty} P(-z_{\alpha/2} \leq Z_K \leq z_{\alpha/2}) = 2 \cdot P(0 \leq Z_K \leq z_{\alpha/2}) = \alpha$$

Note that

$$-z_{\alpha/2} \leq Z_K \leq z_{\alpha/2} \text{ implies that } \left(\hat{\mu} - z_{\alpha/2} \frac{\sigma}{\sqrt{K}} \right) \leq \mu \leq \left(\hat{\mu} + z_{\alpha/2} \frac{\sigma}{\sqrt{K}} \right)$$

We obtain bounds on the value of the true mean μ in terms of the estimated value $\hat{\mu}$:

$$\lim_{K \rightarrow \infty} P \left(\hat{\mu} - z_{\alpha/2} \frac{\sigma}{\sqrt{K}} \leq \mu \leq \hat{\mu} + z_{\alpha/2} \frac{\sigma}{\sqrt{K}} \right) = 1 - \alpha$$

Thus, for any given *level of confidence* α , we can compute the probability that the true mean μ lies in the $\alpha\%$ confidence interval $\left(\hat{\mu} - z_{\alpha/2} \frac{\sigma}{\sqrt{K}}, \hat{\mu} + z_{\alpha/2} \frac{\sigma}{\sqrt{K}} \right)$.

Confidence Intervals: Unknown Variance

The true variance σ^2 is usually unknown, but we may use the sample variance:

$$\hat{\sigma}^2 = \frac{1}{K} \sum_{i=1}^K (\theta_i - \hat{\mu})^2$$

because $\hat{\sigma}^2$ is a *consistent* estimator for σ^2 , that is, as $K \rightarrow \infty$, $\hat{\sigma}^2$ converges with probability 1, also called *converges almost surely*, to σ^2 .

The central limit theorem then states that the random variable Z_K^* defined below converges in distribution to the standard normal distribution:

$$Z_K^* = \sqrt{K} \left(\frac{\hat{\mu} - \mu}{\hat{\sigma}} \right)$$

and thus, we have

$$\lim_{K \rightarrow \infty} P \left(\hat{\mu} - z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{K}} \leq \mu \leq \hat{\mu} + z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{K}} \right) = 1 - \alpha$$

$\left(\hat{\mu} - z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{K}}, \hat{\mu} + z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{K}} \right)$ is the $100(1-\alpha)\%$ confidence interval for μ .

2D Iris Data: Confidence Intervals

Consider the 5-fold cross-validation ($K = 5$) to assess the full Bayes classifier:

$$\hat{\mu}_{\theta} = 0.233 \qquad \hat{\sigma}_{\theta}^2 = 0.00833 \qquad \hat{\sigma}_{\theta} = \sqrt{0.00833} = 0.0913$$

Let $1 - \alpha = 0.95$ be the confidence level and $\alpha = 0.05$ the significance level. The standard normal distribution has 95% of the probability density within $z_{\alpha/2} = 1.96$ standard deviations from the mean.

$$P\left(\mu \in \left(\hat{\mu}_{\theta} - z_{\alpha/2} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}}, \hat{\mu}_{\theta} + z_{\alpha/2} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}}\right)\right) = 0.95$$

Because $z_{\alpha/2} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}} = \frac{1.96 \times 0.0913}{\sqrt{5}} = 0.08$, we have

$$P\left(\mu \in (0.233 - 0.08, 0.233 + 0.08)\right) = P\left(\mu \in (0.153, 0.313)\right) = 0.95$$

With 95% confidence, the true expected error rate lies in the interval $(0.153, 0.313)$. If $\alpha = 0.01$, then $z_{\alpha/2} = 2.58$, $z_{\alpha/2} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}} = \frac{2.58 \times 0.0913}{\sqrt{5}} = 0.105$, and the interval becomes $(0.128, 0.338)$.

Confidence Intervals: Small Sample Size

The confidence interval applies only when the sample size $K \rightarrow \infty$. However, in practice for K -fold cross-validation or bootstrap resampling K is small.

In the small sample case, instead of the normal density to derive the confidence interval, we use the Student's t distribution.

In particular, we choose the value $t_{\alpha/2, K-1}$ such that the cumulative t distribution function with $K - 1$ degrees of freedom encompasses $\alpha/2$ of the probability mass, that is,

$$P(0 \leq Z_K^* \leq t_{\alpha/2, K-1}) = T_{K-1}(t_{\alpha/2}) - T_{K-1}(0) = \alpha/2$$

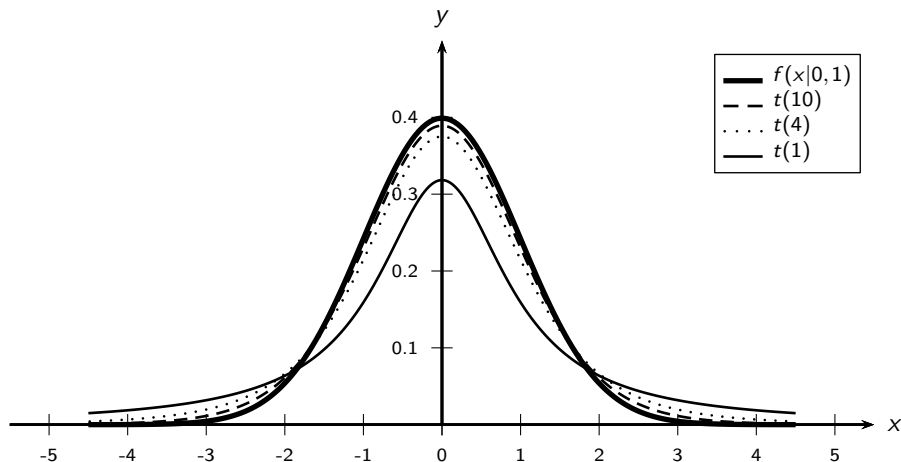
where T_{K-1} is the cumulative distribution function for the Student's t distribution with $K - 1$ degrees of freedom.

The $100(1-\alpha\%)$ confidence interval for the true mean μ is thus

$$\left(\hat{\mu} - t_{\alpha/2, K-1} \frac{\hat{\sigma}}{\sqrt{K}} \leq \mu \leq \hat{\mu} + t_{\alpha/2, K-1} \frac{\hat{\sigma}}{\sqrt{K}} \right)$$

Note the dependence of the interval on both α and the sample size K .

Student's t Distribution: K Degrees of Freedom



Iris 2D Data: Small Sample Confidence Intervals

Due to the small sample size ($K = 5$), we can get a better confidence interval by using the t distribution. For $K - 1 = 4$ degrees of freedom, for $\alpha = 0.95$, we get $t_{\alpha/2, K-1} = 2.776$. Thus,

$$t_{\alpha/2, K-1} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}} = 2.776 \times \frac{0.0913}{\sqrt{5}} = 0.113$$

The 95% confidence interval is therefore

$$(0.233 - 0.113, 0.233 + 0.113) = (0.12, 0.346)$$

which is much wider than the overly optimistic confidence interval $(0.153, 0.313)$ obtained for the large sample case.

For $1 - \alpha = 0.99$, $t_{\alpha/2} = 4.604$, $t_{\alpha/2} \frac{\hat{\sigma}_{\theta}}{\sqrt{K}} = 4.604 \times \frac{0.0913}{\sqrt{5}} = 0.188$ and the 99% confidence interval is

$$(0.233 - 0.188, 0.233 + 0.188) = (0.045, 0.421)$$

This is also much wider than the 99% confidence interval $(0.128, 0.338)$ obtained for the large sample case.

Comparing Classifiers: Paired t -Test

How can we test for a significant difference in the classification performance of two alternative classifiers, M^A and M^B on a given dataset D .

We can apply K -fold cross-validation (or bootstrap resampling) and tabulate their performance over each of the K folds, with identical folds for both classifiers. That is, we perform a *paired test*, with both classifiers trained and tested on the same data.

Let $\theta_1^A, \theta_2^A, \dots, \theta_K^A$ and $\theta_1^B, \theta_2^B, \dots, \theta_K^B$ denote the performance values for M_A and M_B , respectively. To determine if the two classifiers have different or similar performance, define the random variable δ_i as the difference in their performance on the i th dataset:

$$\delta_i = \theta_i^A - \theta_i^B$$

The expected difference and the variance estimates are given as:

$$\hat{\mu}_\delta = \frac{1}{K} \sum_{i=1}^K \delta_i \qquad \hat{\sigma}_\delta^2 = \frac{1}{K} \sum_{i=1}^K (\delta_i - \hat{\mu}_\delta)^2$$

Comparing Classifiers: Paired t -Test

The null hypothesis H_0 is that the performance of M^A and M^B is the same. The alternative hypothesis H_a is that they are not the same, that is:

$$H_0: \mu_\delta = 0$$

$$H_a: \mu_\delta \neq 0$$

Define the z -score random variable for the estimated expected difference as

$$Z_\delta^* = \sqrt{K} \left(\frac{\hat{\mu}_\delta - \mu_\delta}{\hat{\sigma}_\delta} \right)$$

Z_δ^* follows a t distribution with $K - 1$ degrees of freedom. However, under the null hypothesis we have $\mu_\delta = 0$, and thus

$$Z_\delta^* = \frac{\sqrt{K} \hat{\mu}_\delta}{\hat{\sigma}_\delta} \sim t_{K-1}$$

i.e., Z_δ^* follows the t distribution with $K - 1$ degrees of freedom.

Given a desired confidence level α , we conclude that

$$P(-t_{\alpha/2, K-1} \leq Z_\delta^* \leq t_{\alpha/2, K-1}) = \alpha$$

Put another way, if $Z_\delta^* \notin (-t_{\alpha/2, K-1}, t_{\alpha/2, K-1})$, then we may reject the null hypothesis with $\alpha\%$ confidence.

Paired t -Test via Cross-Validation

Paired t -Test(α, K, D):

- 1 $D \leftarrow$ randomly shuffle D
- 2 $\{D_1, D_2, \dots, D_K\} \leftarrow$ partition D in K equal parts
- 3 **foreach** $i \in [1, K]$ **do**
- 4 $M_i^A, M_i^B \leftarrow$ train the two different classifiers on $D \setminus D_i$
- 5 $\theta_i^A, \theta_i^B \leftarrow$ assess M_i^A and M_i^B on D_i
- 6 $\delta_i = \theta_i^A - \theta_i^B$
- 7 $\hat{\mu}_\delta = \frac{1}{K} \sum_{i=1}^K \delta_i$
- 8 $\hat{\sigma}_\delta^2 = \frac{1}{K} \sum_{i=1}^K (\delta_i - \hat{\mu}_\delta)^2$
- 9 $Z_\delta^* = \frac{\sqrt{K} \hat{\mu}_\delta}{\hat{\sigma}_\delta}$
- 10 **if** $Z_\delta^* \in (-t_{\alpha/2, K-1}, t_{\alpha/2, K-1})$ **then**
- 11 Accept H_0 ; both classifiers have similar performance
- 12 **else**
- 13 Reject H_0 ; classifiers have significantly different performance

2D Iris Dataset: Paired t -Test

We compare, via error rate, the naive Bayes (M^A) with the full Bayes (M^B) classifier via cross-validation using $K = 5$.

i	1	2	3	4	5
θ_i^A	0.233	0.267	0.1	0.4	0.3
θ_i^B	0.2	0.2	0.167	0.333	0.233
δ_i	0.033	0.067	-0.067	0.067	0.067

The estimated expected difference and variance of the differences are

$$\hat{\mu}_\delta = \frac{0.167}{5} = 0.033 \quad \hat{\sigma}_\delta^2 = 0.00333 \quad \hat{\sigma}_\delta = \sqrt{0.00333} = 0.0577$$

The z-score value is given as $Z_\delta^* = \frac{\sqrt{K}\hat{\mu}_\delta}{\hat{\sigma}_\delta} = \frac{\sqrt{5} \times 0.033}{0.0577} = 1.28$ For $1 - \alpha = 0.95$ (or $\alpha = 0.05$) and $4(K - 1)$ degrees of freedom, $t_{\alpha/2} = 2.776$.

Because $Z_\delta^* = 1.28 \in (-2.776, 2.776) = (-t_{\alpha/2}, t_{\alpha/2})$, we cannot reject the null hypothesis, that is, there is no significant difference between the naive and full Bayes classifier for this dataset.

Bias-Variance Decomposition

In many applications there may be costs associated with making wrong predictions. A *loss function* specifies the cost or penalty of predicting the class to be $\hat{y} = M(\mathbf{x})$, when the true class is y .

A commonly used loss function for classification is the *zero-one loss*, defined as

$$L(y, M(\mathbf{x})) = I(M(\mathbf{x}) \neq y) = \begin{cases} 0 & \text{if } M(\mathbf{x}) = y \\ 1 & \text{if } M(\mathbf{x}) \neq y \end{cases}$$

Thus, zero-one loss assigns a cost of zero if the prediction is correct, and one otherwise.

Another commonly used loss function is the *squared loss*, defined as

$$L(y, M(\mathbf{x})) = (y - M(\mathbf{x}))^2$$

where we assume that the classes are discrete valued, and not categorical.

Expected Loss

An ideal or optimal classifier is the one that minimizes the loss function. Because the true class is not known for a test case $\mathbf{x}$, the goal of learning a classification model can be cast as minimizing the expected loss:

$$E_y[L(y, M(\mathbf{x})) | \mathbf{x}] = \sum_y L(y, M(\mathbf{x})) \cdot P(y|\mathbf{x})$$

where $P(y|\mathbf{x})$ is the conditional probability of class y given test point $\mathbf{x}$, and E_y denotes that the expectation is taken over the different class values y .

Minimizing the expected zero-one loss corresponds to minimizing the error rate. Let $M(\mathbf{x}) = c_i$, then we have

$$E_y[L(y, M(\mathbf{x})) | \mathbf{x}] = \sum_y I(y \neq c_i) \cdot P(y|\mathbf{x}) = \sum_{y \neq c_i} P(y|\mathbf{x}) = 1 - P(c_i|\mathbf{x})$$

To minimize E_y , we should choose $c_i = \arg \max_y P(y|\mathbf{x})$.

The error rate is an estimate of the expected zero-one loss, which minimizes the error rate.

Bias and Variance

The expected loss for the squared loss function offers important insight into the classification problem because it can be decomposed into bias and variance terms.

Intuitively, the *bias* of a classifier refers to the systematic deviation of its predicted decision boundary from the true decision boundary, whereas the *variance* of a classifier refers to the deviation among the learned decision boundaries over different training sets.

Because M depends on the training set, given a test point $\mathbf{x}$, we denote its predicted value as $M(\mathbf{x}, \mathbf{D})$. Consider the expected square loss:

$$\begin{aligned} E_y \left[L(y, M(\mathbf{x}, \mathbf{D})) \mid \mathbf{x}, \mathbf{D} \right] &= E_y \left[(y - M(\mathbf{x}, \mathbf{D}))^2 \mid \mathbf{x}, \mathbf{D} \right] \\ &= \underbrace{E_y \left[(y - E_y[y \mid \mathbf{x}])^2 \mid \mathbf{x}, \mathbf{D} \right]}_{\text{var}(y \mid \mathbf{x})} + \underbrace{\left(M(\mathbf{x}, \mathbf{D}) - E_y[y \mid \mathbf{x}] \right)^2}_{\text{squared-error}} \end{aligned}$$

The first term is simply the variance of y given $\mathbf{x}$. The second term is the squared error between the predicted value $M(\mathbf{x}, \mathbf{D})$ and the expected value $E_y[y \mid \mathbf{x}]$.

Bias and Variance

The squared error depends on the training set. We can eliminate this dependence by averaging over all possible training tests of size n . The average or expected squared error for a given test point x over all training sets is then given as

$$E_D \left[(M(x, D) - E_y[y|x])^2 \right] = \underbrace{E_D \left[(M(x, D) - E_D[M(x, D)])^2 \right]}_{\text{variance}} + \underbrace{\left(E_D[M(x, D)] - E_y[y|x] \right)^2}_{\text{bias}}$$

The expected squared loss over all test points x and over all training sets D of size n yields the following decomposition:

$$\begin{aligned} E_{x,D,y} \left[(y - M(x, D))^2 \right] &= \underbrace{E_{x,y} \left[(y - E_y[y|x])^2 \right]}_{\text{noise}} + \underbrace{E_{x,D} \left[(M(x, D) - E_D[M(x, D)])^2 \right]}_{\text{average variance}} \\ &\quad + \underbrace{E_x \left[(E_D[M(x, D)] - E_y[y|x])^2 \right]}_{\text{average bias}} \end{aligned}$$

The expected square loss over all test points and training sets can be decomposed into three terms: noise, average bias, and average variance.

Bias and Variance

The noise term is the average variance $\text{var}(y|\mathbf{x})$ over all test points $\mathbf{x}$. It contributes a fixed cost to the loss independent of the model, and can thus be ignored when comparing different classifiers.

The classifier specific loss can then be attributed to the variance and bias terms. Bias indicates whether the model M is correct or incorrect.

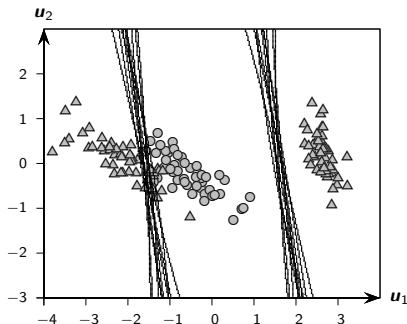
If the decision boundary is nonlinear, and we use a linear classifier, then it is likely to have high bias. A nonlinear (or a more complex) classifier is more likely to capture the correct decision boundary, and is thus likely to have a low bias.

The complex classifier is not necessarily better, since we also have to consider the variance term, which measures the inconsistency of the classifier decisions. A complex classifier induces a more complex decision boundary and thus may be prone to *overfitting*, and thus may be susceptible to small changes in training set, which may result in high variance.

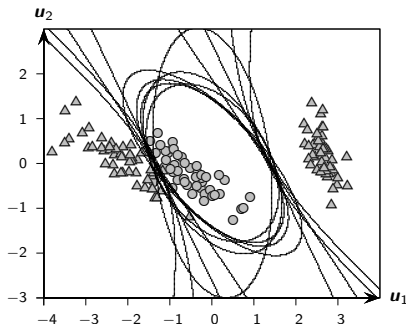
In general, the expected loss can be attributed to high bias or high variance, typically a trade-off between the two. We prefer a classifier with an acceptable bias and as low a variance as possible.

Bias-variance Decomposition: SVM Quadratic Kernels

Iris PC Data: Iris-versicolor (c_1 -circles) and other two Irises (c_2 - triangles).
 $K = 10$ Bootstrap samples, trained via SVMs, varying the regularization constant C from 10^{-2} to 10^2 . The decision boundaries over the 10 samples were as follows:



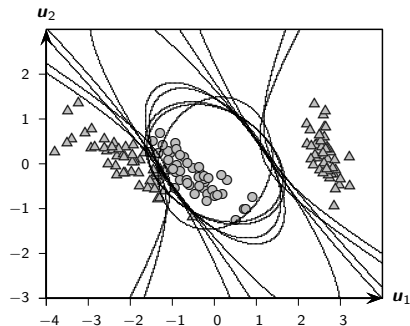
(a) $C = 0.01$



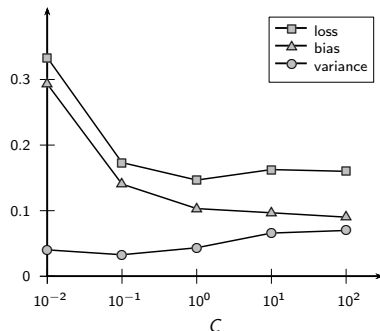
(b) $C = 1$

A small value of C emphasizes the margin, whereas a large value of C tries to minimize the slack terms.

Bias-variance Decomposition: SVM Quadratic Kernels



(c) $C = 100$



(d) Bias-Variance

Variance of the SVM model increases as we increase C .

The bias-variance tradeoff is visible, since as the bias reduces, the variance increases.

The lowest expected loss is obtained when $C = 1$.

Ensemble Classifiers

A classifier is called *unstable* if small perturbations in the training set result in large changes in the prediction or decision boundary.

High variance classifiers are inherently unstable, since they tend to overfit the data. On the other hand, high bias methods typically underfit the data, and usually have low variance.

In either case, the aim of learning is to reduce classification error by reducing the variance or bias, ideally both.

Ensemble methods create a *combined classifier* using the output of multiple *base classifiers*, which are trained on different data subsets. Depending on how the training sets are selected, and on the stability of the base classifiers, ensemble classifiers can help reduce the variance and the bias, leading to a better overall performance.

Bagging

Bagging stands for *Bootstrap Aggregation*. It is an ensemble classification method that employs multiple bootstrap samples (with replacement) from the input training data $\mathbf{D}$ to create slightly different training sets $\mathbf{D}_i$, $i = 1, 2, \dots, K$. Different base classifiers M_i are learned, with M_i trained on $\mathbf{D}_i$.

Given any test point $\mathbf{x}$, it is first classified using each of the K base classifiers, M_i . Let the number of classifiers that predict the class of $\mathbf{x}$ as c_j be given as

$$v_j(\mathbf{x}) = \left| \{ M_i(\mathbf{x}) = c_j \mid i = 1, \dots, K \} \right|$$

The combined classifier, denoted $\mathbf{M}^K$, predicts the class of a test point $\mathbf{x}$ by *majority voting* among the k classes:

$$\mathbf{M}^K(\mathbf{x}) = \arg \max_{c_j} \left\{ v_j(\mathbf{x}) \mid j = 1, \dots, k \right\}$$

Bagging can help reduce the variance, especially if the base classifiers are unstable, due to the averaging effect of majority voting. It does not, in general, have much effect on the bias.

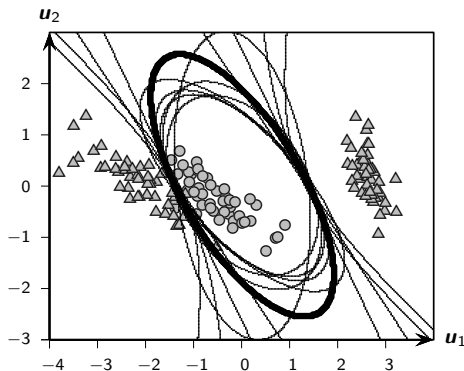
Bagging: Combined SVM Classifiers

SVM classifiers are trained on $K = 10$ bootstrap samples of the Iris PCA dataset using $C = 1$.

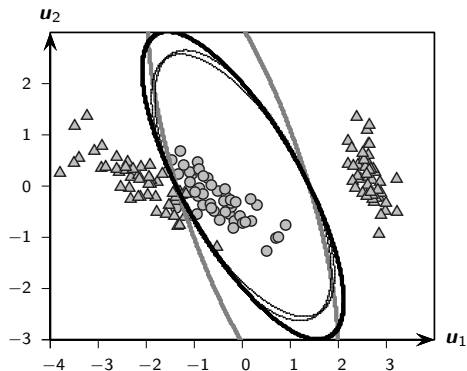
K	Zero-one loss	Squared loss
3	0.047	0.187
5	0.04	0.16
8	0.02	0.10
10	0.027	0.113
15	0.027	0.107

Bagging: Combined SVM Classifiers

The combined (average) classifier is shown in bold.



(a) $K = 10$



(b) Effect of K

The worst training performance is obtained for $K = 3$ (in thick gray) and the best for $K = 8$ (in thick black).

Random Forest

A *random forest* is an ensemble of K classifiers, $M_1, \dots, M_K$, where each classifier is a decision tree created from a different bootstrap sample, and by sampling a random subset of the attributes at each internal node in the decision tree, typically $\sqrt{d}$.

Bagging only would generate similar decision trees. The attribute sampling reduces the trees correlation.

The K decision trees $M_1, M_2, \dots, M_K$ predict the class of $\mathbf{x}$ by majority voting:

$$\mathbf{M}^K(\mathbf{x}) = \arg \max_{\mathcal{G}} \left\{ v_j(\mathbf{x}) \mid j = 1, \dots, k \right\}$$

where v_j is the number of trees that predict the class of $\mathbf{x}$ as $\mathcal{G}_j$.

Notice that if $p = d$ then the random forest approach is equivalent to bagging over decision tree models.

Random Forest Algorithm

RandomForest(D, K, p, η, π):

```
1 foreach  $\mathbf{x}_i \in D$  do
2    $v_j(\mathbf{x}_i) \leftarrow 0$ , for all  $j = 1, 2, \dots, k$ 
3 for  $t \in [1, K]$  do
4    $D_t \leftarrow$  sample of size  $n$  with replacement from  $D$ 
5    $M_t \leftarrow$  DecisionTree( $D_t, \eta, \pi, p$ )
6   foreach  $(\mathbf{x}_i, y_i) \in D \setminus D_t$  do // out-of-bag votes
7      $\hat{y}_i \leftarrow M_t(\mathbf{x}_i)$ 
8     if  $\hat{y}_i = c_j$  then  $v_j(\mathbf{x}_i) = v_j(\mathbf{x}_i) + 1$ 
9    $\epsilon_{oob} = \frac{1}{n} \cdot \sum_{i=1}^n I\left(y_i \neq \arg \max_{\mathcal{G}} \{v_j(\mathbf{x}_i) | (\mathbf{x}_i, y_i) \in D\}\right)$  // OOB
    error
10 return  $\{M_1, M_2, \dots, M_K\}$ 
```

Random Forest - Out of bag estimation

Given $\mathbf{D}_t$, any point in $\mathbf{D} \setminus \mathbf{D}_t$ is called an *out-of-bag* point for M_t .

The *out-of-bag* error rate for each M_t may be calculated by considering the prediction over its out-of-bag points.

The out-of-bag (OOB) error for the random forest is given as:

$$\epsilon_{oob} = \frac{1}{n} \cdot \sum_{i=1}^n I\left(y_i \neq \max_{\mathcal{G}} \{v_j(\mathbf{x}_i) | (\mathbf{x}_i, y_i) \in \mathbf{D}\}\right)$$

I is an indicator function that returns 1 if its argument is true, and 0 otherwise.

We check whether the majority OOB class for $\mathbf{x}_i \in \mathbf{D}$ matches y_i .

The out-of-bag error rate is the fraction of mismatched points w.r.t. y_i .

The out-of-bag error rate approximates the cross-validation error rate quite well.

Iris PC Dataset: Random Forest

The task is to separate Iris-versicolor (c_1 - circles) from the other two Irises (c_2 - triangles).

Since $d = 2$, we pick $p = 1$ attribute for each split-point evaluation.

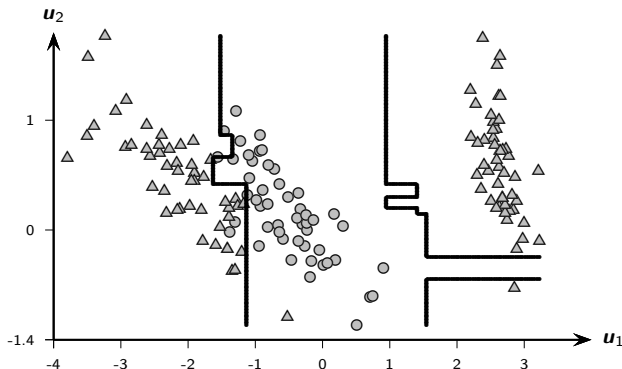
We use $\eta = 3$ (maximum leaf size) and minimum purity $\pi = 1.0$.

We grow $K = 5$ decision trees on different bootstrap samples.

Iris PC Dataset: Random Forest

Decision boundary is shown in bold.

The training error rate is 2.0%, but OOB error rate is 49.33%, which is overly pessimistic in this case, since the dataset has only two attributes, and we use only one attribute to evaluate each split point.



Random Forest: Varying K

We used the full Iris dataset which has four attributes ($d = 4$), and three classes ($k = 3$).

We employed $p = \sqrt{d} = 2$, $\eta = 3$ and $\pi = 1.0$.

K	ϵ_{oob}	ϵ
1	0.4333	0.0267
2	0.2933	0.0267
3	0.1867	0.0267
4	0.1200	0.0400
5	0.1133	0.0333
6	0.1067	0.0400
7	0.0733	0.0333
8	0.0600	0.0267
9	0.0467	0.0267
10	0.0467	0.0267

We can see that the OOB error decreases as we increase the number of trees.

In *Boosting* the main idea is to carefully select the samples to *boost* the performance on hard to classify instances.

Starting from an initial training sample D_1 , we train the base classifier M_1 , and obtain its training error rate.

To construct the next sample D_2 , we select the misclassified instances with higher probability, and after training M_2 , we obtain its training error rate.

To construct D_3 , those instances that are hard to classify by M_1 or M_2 , have a higher probability of being selected. This process is repeated for K iterations.

Finally, the combined classifier is obtained via weighted voting over the output of the K base classifiers $M_1, M_2, \dots, M_K$.

Boosting is most beneficial when the base classifiers are *weak*, that is, have an error rate that is slightly less than that for a random classifier.

The idea is that whereas M_1 may not be particularly good on all test instances, by design M_2 may help classify some cases where M_1 fails, and M_3 may help classify instances where M_1 and M_2 fail, and so on. Thus, boosting has more of a bias reducing effect.

Each of the weak learners is likely to have high bias (it is only slightly better than random guessing), but the final combined classifier can have much lower bias, since different weak learners learn to classify instances in different regions of the input space.

Adaptive Boosting: AdaBoost

AdaBoost repeats the boosting process K times. Let t denote the iteration and let α_t denote the weight for the t th classifier M_t .

Let w_i^t denote the weight for $\mathbf{x}_i$, with $\mathbf{w}^t = (w_1^t, w_2^t, \dots, w_n^t)^T$ being the weight vector over all the points for the t th iteration.

$\mathbf{w}$ is a probability vector, whose elements sum to one. Initially all points have equal weights, that is,

$$\mathbf{w}^0 = \left(\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n} \right)^T = \frac{1}{n} \mathbf{1}$$

During iteration t , the training sample $\mathbf{D}_t$ is obtained via weighted resampling using the distribution $\mathbf{w}^{t-1}$, that is, we draw a sample of size n with replacement, such that the i th point is chosen according to its probability w_i^{t-1} .

Using $\mathbf{D}_t$ we train the classifier M_t , and compute its weighted error rate ϵ_t on the entire input dataset $\mathbf{D}$:

$$\epsilon_t = \sum_{i=1}^n w_i^{t-1} \cdot I(M_t(\mathbf{x}_i) \neq y_i)$$

Adaptive Boosting: AdaBoost

The weight for the t th classifier is then set as

$$\alpha_t = \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

The weight for each point $\mathbf{x}_i \in \mathbf{D}$ is updated as

$$w_i^t = w_i^{t-1} \cdot \exp \left\{ \alpha_t \cdot I(M_t(\mathbf{x}_i) \neq y_i) \right\}$$

If the predicted class matches the true class, that is, if $M_t(\mathbf{x}_i) = y_i$, then the weight for point $\mathbf{x}_i$ remains unchanged.

If the point is misclassified, that is, $M_t(\mathbf{x}_i) \neq y_i$, then

$$w_i^t = w_i^{t-1} \cdot \exp \{ \alpha_t \} = w_i^{t-1} \exp \left\{ \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \right\} = w_i^{t-1} \left(\frac{1}{\epsilon_t} - 1 \right)$$

Thus, if the error rate ϵ_t is small, then there is a greater weight increment for $\mathbf{x}_i$. The intuition is that a point that is misclassified by a good classifier (with a low error rate) should be more likely to be selected for the next training dataset.

Adaptive Boosting: AdaBoost

For boosting we require that a base classifier has an error rate at least slightly better than random guessing, that is, $\epsilon_t < 0.5$. If the error rate $\epsilon_t \geq 0.5$, then the boosting method discards the classifier, and tries another data sample.

Combined Classifier: Given the set of boosted classifiers, $M_1, M_2, \dots, M_K$, along with their weights $\alpha_1, \alpha_2, \dots, \alpha_K$, the class for a test case $\mathbf{x}$ is obtained via weighted majority voting.

Let $v_j(\mathbf{x})$ denote the weighted vote for class c_j over the K classifiers, given as

$$v_j(\mathbf{x}) = \sum_{t=1}^K \alpha_t \cdot I(M_t(\mathbf{x}) = c_j)$$

Because $I(M_t(\mathbf{x}) = c_j)$ is 1 only when $M_t(\mathbf{x}) = c_j$, the variable $v_j(\mathbf{x})$ simply obtains the tally for class c_j among the K base classifiers, taking into account the classifier weights. The combined classifier, denoted $\mathbf{M}^K$, then predicts the class for $\mathbf{x}$ as follows:

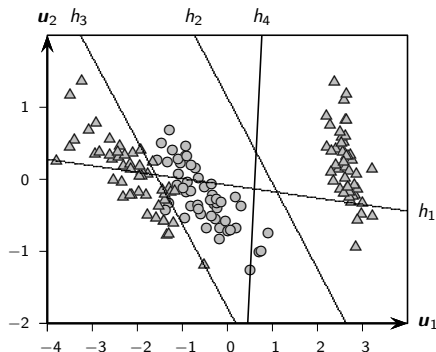
$$\mathbf{M}^K(\mathbf{x}) = \arg \max_{c_j} \left\{ v_j(\mathbf{x}) \mid j = 1, \dots, k \right\}$$

AdaBoost Algorithm

AdaBoost(K, D):

```
1  $\mathbf{w}^0 \leftarrow \left(\frac{1}{n}\right) \cdot \mathbf{1} \in \mathbb{R}^n$ 
2  $t \leftarrow 1$ 
3 while  $t \leq K$  do
4    $D_t \leftarrow$  weighted resampling with replacement from  $D$  using  $\mathbf{w}^{t-1}$ 
5    $M_t \leftarrow$  train classifier on  $D_t$ 
6    $\epsilon_t \leftarrow \sum_{i=1}^n w_i^{t-1} \cdot I(M_t(\mathbf{x}_i) \neq y_i)$  // weighted error rate on  $D$ 
7   if  $\epsilon_t = 0$  then break
8   else if  $\epsilon_t < 0.5$  then
9      $\alpha_t = \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$  // classifier weight
10    foreach  $i \in [1, n]$  do
11      // update point weights
12       $w_i^t = \begin{cases} w_i^{t-1} & \text{if } M_t(\mathbf{x}_i) = y_i \\ w_i^{t-1} \left(\frac{1-\epsilon_t}{\epsilon_t}\right) & \text{if } M_t(\mathbf{x}_i) \neq y_i \end{cases}$ 
13     $\mathbf{w}^t = \frac{\mathbf{w}^t}{\mathbf{1}^T \mathbf{w}^t}$  // normalize weights
14     $t \leftarrow t + 1$ 
15  return  $\{M_1, M_2, \dots, M_K\}$ 
```

Boosting SVMs: Linear Kernel ($C = 1$)



The regularization constant $C = 1$.

h_t is learned in iteration t .

No h_t discriminates between classes well:

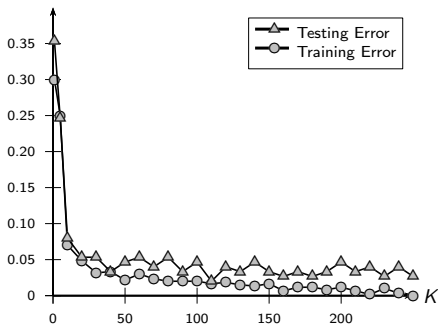
M_t	ϵ_t	α_t
h_1	0.280	0.944
h_2	0.305	0.826
h_3	0.174	1.559
h_4	0.282	0.935

When we combine the decisions from hyperplanes weighted by α_t , we observe a marked drop in the error rate:

combined model	M^1	M^2	M^3	M^4
training error rate	0.280	0.253	0.073	0.047

Boosting SVMs: Linear Kernel ($C = 1$)

Five-fold cross-validation using independent testing data.



As the number of base classifiers K increases, both error rates reduce.

However, while the training error essentially goes to 0, the testing error does not reduce beyond 0.02 ($K = 110$).

This example illustrates the effectiveness of boosting in reducing the bias.

Bagging can be considered as a special case of AdaBoost, where $w^t = \frac{1}{n}1$, and $\alpha_t = 1$ for all K iterations.

The weighted resampling defaults to regular resampling with replacement, and majority voting predicts the class.

Stacking

Stacking or stacked generalization is an ensemble technique where we employ two layers of classifiers.

The first layer is composed of K base classifiers which are trained independently on the entire training data D .

The second layer comprises a combiner classifier C that is trained on the predicted classes from the base classifiers, so that it automatically learns how to combine the outputs of the base classifiers to make the final prediction for a given input.

For example, the combiner classifier may learn to ignore the output of a base classifiers for an input that lies in a region of the input space where the base classifier has poor performance. It can also learn to correct the prediction in cases where most base classifiers do not predict the outcome correctly.

Stacking is a strategy for estimating and correcting the biases of the set of base classifiers.

```
Stacking( $K, \mathbf{M}, C, \mathbf{D}$ ):  
  // Train base classifiers  
1 for  $t \in [1, K]$  do  
2    $M_t \leftarrow$  train  $t$ th base classifier on  $\mathbf{D}$   
  // Train combiner model  $C$  on  $\mathbf{Z}$   
3  $\mathbf{Z} \leftarrow \emptyset$   
4 foreach  $(\mathbf{x}_i, y_i) \in \mathbf{D}$  do  
5    $\mathbf{z}_i \leftarrow (M_1(\mathbf{x}_i), M_2(\mathbf{x}_i), \dots, M_K(\mathbf{x}_i))^T$   
6    $\mathbf{Z} \leftarrow \mathbf{Z} \cup \{(\mathbf{z}_i, y_i)\}$   
7  $C \leftarrow$  train combiner classifier on  $\mathbf{Z}$   
8 return  $(C, M_1, M_2, \dots, M_K)$ 
```

Iris 2D PCA - Stacking

We use three base classifiers:

- SVM with a linear kernel (with regularization constant $C = 1$),
- random forests (with $K = 5$ trees and $p = 1$ random attributes), and
- naive Bayes.

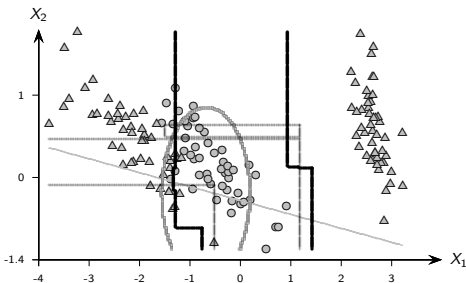
The combiner classifier is an SVM with a Gaussian kernel (with regularization constant $C = 1$ and spread parameter $\sigma^2 = 0.2$).

Training uses an 100-point random subset.

Testing uses the remaining 50 points.

Classifier	Test Accuracy
Linear SVM	0.68
Random Forest	0.82
Naive Bayes	0.74
Stacking	0.92

Iris 2D PCA - Stacking



Plot shows the decision boundaries for each of the three base classifiers:

- linear SVM boundary is the line in light gray,
- naive Bayes boundary is the ellipse comprising the gray circles, and
- random forest boundary is shown via plusses ('+').

The boundary of the stacking classifier is shown as the thicker black lines.

Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 22: Classification Assessment